
1. Difference Between let and var

Concept:

- var is **function-scoped** and gets **hoisted** (declared at top of scope).
- let is **block-scoped** and does **not hoist** in the same way (it's in the temporal dead zone).

Example:

```
{  
  var a = 10;  
  let b = 20;  
}  
console.log(a); // 10  
console.log(b); // ReferenceError
```

Real-life analogy:

Think of let as a keycard to a **room** (block), while var is access to the **entire building** (function).

2. Difference and Similarities Between let and const

Similarities:

- Both are block-scoped.
- Not hoisted like var.

Differences:

- let allows reassignment.
- const doesn't allow reassignment (must be initialized).

Example:

```
let x = 5;  
x = 10; // OK
```

```
const y = 5;  
y = 10; // Error
```

Real-life analogy:

let is like a locker you can reopen and change the contents, const is like sealing it forever.

3. Why use 'use strict'?

Concept:

- Enables strict mode in JS.
- Catches errors like undeclared variables.
- Prevents usage of reserved keywords.

Example:

```
"use strict";  
x = 5; // Error: x is not declared
```

Real-life analogy:

It's like turning on **safe mode** while coding.

4. What is the use of typeof?

Concept:

- Tells the **data type** of a value or variable.

Example:

```
typeof "hello" // "string"  
typeof 42      // "number"
```

Real-life analogy:

It's like checking a **label** on a package to know what's inside.

5. Which operator is overloaded?

Answer:

- + operator. It works for both **addition** and **string concatenation**.

Example:

```
1 + 2    // 3  
"Hi " + "JS" // "Hi JS"
```

6. Default value if variable is not assigned?

Answer: undefined

Example:

```
let x;  
console.log(x); // undefined
```

7. Calculation between number and undefined results in?

Answer: NaN (Not a Number)

Example:

```
let x = 5 + undefined; // NaN
```

8. Default value returned by function?

Answer: undefined

Example:

```
function greet() {}  
console.log(greet()); // undefined
```

9. Explain window.document.write()

Concept:

- Writes content directly to the web page from script.
- Can **overwrite** the whole HTML if used after loading.

Example:

```
window.document.write("Hello");
```

10. Is JavaScript object-oriented?

Answer:

- **Yes**, JavaScript supports object-oriented programming using **prototypes** and **classes (ES6)**.
-

11. List predefined references [Object] in JS

Examples:

- Object
- Array

- Function
- Date
- RegExp
- String
- Number
- Boolean
- Math
- JSON

12. Which version you have learned?

Suggested Answer:

"I've learned **ES6 and above**, which includes modern features like let/const, arrow functions, promises, classes, etc."

13. Why JS is called client-side?

Answer:

- JavaScript runs in the **browser**, on the client's machine, not on the server by default.

14. Is JavaScript dynamic? Explain with example.

Answer:

- Yes, it's dynamically typed: variable types are determined at runtime.

Example:

```
let x = 5;  
x = "now a string"; // No error
```

15. Do parameters in function call and definition need to be same?

Answer:

- No, extra arguments are ignored, and missing ones become undefined.

Example:

```
function test(a, b) {  
  console.log(a, b);  
}  
test(1); // 1 undefined  
test(1, 2, 3); // 1 2
```

16. Do we need to use ; semicolon in JS?

Answer:

- Optional, due to **automatic semicolon insertion (ASI)**, but it's **recommended** for consistency.

17. Use of JavaScript?

Answer:

- Making web pages interactive.
- Form validation.
- DOM manipulation.
- Backend (Node.js).

- Game development, mobile apps, etc.
-

18. Frameworks or Libraries built using JS

Examples:

- React
 - Angular
 - Vue
 - Node.js
 - Express
 - jQuery
-

19. How to get count of a given string

Answer:

Use .length

Example:

```
let str = "hello";  
console.log(str.length); // 5
```

20. What is valueOf() and where is it defined?

Concept:

- It returns the **primitive value** of an object.
- Defined in Object, Number, String, etc.

Example:

```
let num = new Number(42);  
console.log(num.valueOf()); // 42
```

21. Difference between rest and spread

Concept:

- **Rest ...**: Combines items into an array (in functions).
- **Spread ...**: Expands items from array/object.

Example:

```
function add(...nums) { return nums; } // Rest  
let arr = [1, 2];  
let newArr = [...arr, 3]; // Spread
```

22. How to clone an object

Ways:

1. Using spread:

```
let clone = { ...original };
```

2. Using Object.assign():

```
let clone = Object.assign({}, original);
```

3. Deep clone (for nested objects):

```
let deepClone = JSON.parse(JSON.stringify(obj));
```

✓ Use of == and ===

- == → Compares **value** (type coercion allowed)
- === → Compares **value + type** (strict)

```
var a = 5;  
let b = '5';
```

```
console.log(a == b); // true  
console.log(a === b); // false
```

Analogy:

== is like comparing "5" and 5 loosely. === checks both type and value like a strict identity check.

✓ What is NaN?

- Means “**Not a Number**”
- Result of invalid numeric operations

```
let x = 0 / 0; // NaN  
isNaN("abc"); // true
```

✓ What is false in JS? (Falsy values)

These are falsy:

- false
- 0
- null
- undefined
- NaN
- "" (empty string)

```
if (0) console.log("Won't run");
```

✓ Different ways to create an object

1. **Object literal**

```
let obj = {name: "JS"};
```

2. **Constructor function**

```
3. function Car() {  
4.   this.type = "SUV";  
5. }
```

```
6. let car = new Car();
```

7. **Object.create()**

```
8. let base = {color: "red"};  
9. let child = Object.create(base);
```

10. **ES6 Class**

```
11. class Person {  
12.   constructor(name) {  
13.     this.name = name;  
14.   }  
15. }
```

✓ **Can you call a class (ES6) without new?**

No, will throw **TypeError**, unless **not in strict mode**, but still not valid behavior.

```
class A {}
```

```
A(); // TypeError: Class constructor A cannot be invoked without 'new'
```

✓ **What is constructor keyword?**

- Special method in class to **initialize object**.

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

✓ **How to override toString() method**

```
let obj = {  
  name: "Test",  
  toString() {  
    return `Name: ${this.name}`;  
  }  
};  
console.log(obj.toString());
```

✓ **Does JS support method overloading?**

No, JS does **not support** traditional overloading. Last defined method wins.

✓ **How to inherit in ES5**

```
function Parent(name) {  
  this.name = name;  
}  
Parent.prototype.sayHi = function() {  
  console.log("Hi");  
};  
  
function Child(name) {  
  Parent.call(this, name); // like `super()`  
}  
Child.prototype = Object.create(Parent.prototype);
```

✓ **Everything is derived from?**

→ Object

✓ **Top level class in browser**

→ Window object

✓ **Iterate properties of an object**

```
for (let key in obj) {
```

```
console.log(key, obj[key]);  
}
```

✅ Iterate over an array

```
let arr = [1, 2, 3];  
for (let i of arr) console.log(i);
```

✅ What is Closure?

- Inner function that **remembers** variables from outer scope.

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}  
let counter = outer();  
counter(); // 1
```

✅ What is => (Arrow function)?

- Short syntax
- Does **not bind its own this**

```
const greet = () => console.log("Hi");
```

✅ What is IIFE (Immediately Invoked Function Expression)?

```
(function() {  
  console.log("IIFE runs immediately");  
})();
```

✅ What is default argument?

```
function greet(name = "Guest") {  
  console.log("Hi", name);  
}  
greet(); // Hi Guest
```

✅ What is variable length of argument (...args)

```
function sum(...nums) {  
  return nums.reduce((a, b) => a + b);  
}
```

✅ Prototype inheritance vs extends

```
// ES5  
Child.prototype = Object.create(Parent.prototype);
```

```
// ES6  
class Child extends Parent {}
```

✓ **Parent.call(this, a, b) vs super(a, b)**

- Parent.call(...) → ES5 inheritance
- super(...) → ES6 class inheritance

✓ **# in class → Private field**

```
class Example {  
  #secret = 123;  
}
```

✓ **_ → Convention for Protected (not enforced)**

```
class Example {  
  _semiPrivate = "for subclass";  
}
```

✓ **Static Method**

```
class MyClass {  
  static greet() {  
    console.log("Static hello");  
  }  
}  
MyClass.greet();
```

✓ **Static Variable**

Not directly supported in old JS, but possible:

```
class Counter {  
  static count = 0;  
}
```

✓ **Class vs Function as Class**

- **Class:** clear, strict, constructor, extends, super
- **Function:** used in ES5, less structured

✓ **Method is not enumerable in class**

Yes, class methods are non-enumerable by default.

✓ **Import**

```
import { something } from './module.js';
```

✓ **Export and export default**

```
export const a = 10;  
export default function greet() {}
```

✓ **Node.js**

JavaScript runtime on the **server** using Chrome's V8 engine.

✓ **.mjs**

Used for **ES Modules** in Node.

✓ **Use of this in static method?**

You can, but this refers to the class, **not instance**.

✓ **Best way to declare array**

let arr = []; // ✓ Recommended

Avoid:

new Array(); // ✗ Confusing behavior

✓ **Avoid new Number, new String, new Image**

They create **object wrappers**, not primitives.

✓ **Which method overridden in Array class?**

→ toString() method

[1, 2].toString(); // "1,2"

✓ **new Array(2) means?**

→ Creates array of length 2 with **empty slots**

new Array(2); // [empty × 2]

✓ **length / name**

- length: number of elements
 - name: function or class name
-

✓ **Concat two arrays**

let arr1 = [1];

let arr2 = [2];

let combined = arr1.concat(arr2); // [1, 2]

✓ **Convert array to string**

arr.join(", "); // returns comma-separated string

✓ **Array methods**

- slice(start, end) – extract portion
 - splice(index, deleteCount) – remove/replace
 - push() – add to end
 - pop() – remove last
 - shift() – remove first
 - unshift() – add to start
 - sort() – sort array
 - reverse() – reverse order
-

✓ Difference between find() and filter()

Feature	find()	filter()
Returns	First matching value	All matching values in array
Return type	Single element (or undefined)	Array
Stops at	First match	Goes through entire array

Example:

```
let arr = [1, 2, 3, 4];  
arr.find(x => x > 2); // 3  
arr.filter(x => x > 2); // [3, 4]
```

✓ Difference between arr[i] and arr.at(i)

- arr[i] – Traditional way to access index
- arr.at(i) – Supports **negative indexing** (ES2022)

Example:

```
arr.at(-1); // Last element
```

✓ What is map() function of Array class

- Creates **new array** by transforming elements using a callback.

```
[1, 2, 3].map(x => x * 2); // [2, 4, 6]
```

✓ Return type of includes() method of Array class

→ Returns true or false

```
[1, 2].includes(2); // true
```

✓ Difference between getElementById and getElementsByTagName

Method	Returns	Selector Type
getElementById	Single Element	By ID
getElementsByTagName	HTMLCollection	By Tag

✓ classList methods

```
element.classList.add("className");  
element.classList.remove("className");  
element.classList.replace("old", "new");  
element.classList.toggle("active");  
element.classList.contains("active"); // true/false
```

✓ How to set and get attributes

```
element.setAttribute("href", "https://example.com");  
element.getAttribute("href");
```

✓ Difference between querySelector and querySelectorAll

- querySelector → First matching element
- querySelectorAll → All matching elements (NodeList)

Selectors supported:

- #id, .class, tag, [attr], >, ~, *

```
document.querySelectorAll("div.box");
document.querySelector("#main");
```

✓ Convert string to Date

```
new Date("2023-12-12"); // Date object
```

✓ Generate random number

```
Math.random(); // 0 (inclusive) to <1
```

Range example:

```
let rand = Math.floor(Math.random() * 10); // 0-9
```

✓ String to number

```
parseInt("123"); // 123
parseFloat("12.3"); // 12.3
+"55"; // 55
new Number("55"); // [Number: 55]
```

✓ map, at, find, filter

These are all **Array methods**:

- map() → transform elements
 - at() → get element by index (supports negative)
 - find() → first match
 - filter() → all matches
-

✓ Forms: forms, elements, options

```
document.forms[0];
document.forms[0].elements["username"];
document.forms[0].elements["select1"].options;
```

✓ document.frm.t1

Assuming:

```
<form name="frm">
  <input name="t1">
</form>
```

→ Access:

```
document.frm.t1.value;
```

✓ checked → radio / checkbox

```
document.getElementById("chk").checked;
```

✓ selectedIndex for dropdown

```
let index = document.getElementById("mySelect").selectedIndex;
```

✓ value vs text

```
element.value; // form value
```

element.textContent; // for general text

✓ **onsubmit is for...?**

→ form element

<form onsubmit="return validate();">

- If onsubmit returns false, form submission is **cancelled**

✓ **submit() method**

→ Programmatically submits form

form.submit();

✓ **Set cursor on form field → focus()**

document.getElementById("input1").focus();

✓ **Common Events**

- onchange – Value changed
- onblur – Lost focus
- onerror – Error in script/load
- oninput – User typing (live)
- onkeypress, onkeydown, onkeyup – Keyboard events
- onmouseover, onmouseout
- onmouseenter, onmouseleave
- onclick – Mouse click

✓ **addEventListener() / removeEventListener()**

element.addEventListener("click", myFunc);

element.removeEventListener("click", myFunc);

✓ **event object**

```
element.addEventListener("click", function(e) {  
  console.log(e.target);    // Actual clicked element  
  console.log(e.currentTarget); // Listener attached element  
  console.log(e.type);      // "click"  
});
```

✓ **history object**

history.back();

history.forward();

history.go(-1); // like back

✓ **navigator object**

Contains info about browser/device:

navigator.userAgent;

navigator.geolocation;

✓ **Different ways to fire event**

```
// Inline
<button onclick="go()">
```

```
// With JS
element.onclick = go;
```

```
// Recommended
element.addEventListener("click", go);
```

✅ try-catch-finally

```
try {
  // code
} catch (err) {
  console.log(err.name); // TypeError, ReferenceError, etc.
} finally {
  console.log("Always runs");
}
```

✅ Error types in JS

- TypeError → Wrong type
 - ReferenceError → Undeclared variable
 - URIError → Bad URI
 - SyntaxError → Bad syntax
 - EvalError → eval() issues
 - RangeError → Out of bounds
 - Error → Base class
-

✅ Optional Chaining (?.)

It allows safe access to deeply nested properties or methods **without throwing an error** if any part is undefined or null.

```
obj?.prop
obj.address?.street
obj.method?.()
array?.[1]
obj?.[key]
```

Real-life analogy:

It's like asking: "If the mailbox exists, give me the letter. If not, don't crash."

✅ Nullish Coalescing (??)

Returns the **right-hand value only if the left is null or undefined** (not for 0 or empty string).

```
null ?? "hello"    // "hello"
false ?? "hello"   // false
```

Use case: Provide a default value **only when something is really missing**, not just falsy.

✓ What is AJAX?

AJAX (Asynchronous JavaScript and XML) enables web pages to **send and receive data in the background** without reloading.

✓ XMLHttpRequest (XHR) Basics

```
let xhr = new XMLHttpRequest();  
xhr.open("GET", "data.json", true); // Method to hit server  
xhr.send(); // Sends the request
```

✓ Which method hits the server?

→ .send()
Used after .open() to send the request.

✓ Common XHR Events

- onreadystatechange: Triggers at every state change
- onload: Triggers once when data is fully loaded

```
xhr.onload = function() {  
  console.log(xhr.responseText); // or responseXML  
};
```

✓ XHR Response Types

- responseText → plain text or JSON as string
 - responseXML → parsed XML if applicable
 - response → used in newer APIs to handle binary, JSON, etc.
-

✓ What is a Promise?

A Promise is a JavaScript object that represents the **eventual completion (or failure)** of an asynchronous operation.

```
let promise = new Promise((resolve, reject) => {  
  if (success) resolve("Done!");  
  else reject("Error!");  
});
```

✓ States of a Promise

1. **Pending** – initial state
 2. **Fulfilled** – operation completed successfully (resolve)
 3. **Rejected** – operation failed (reject)
-

✓ then() has how many parameters?

- Two parameters:
 - promise.then(successCallback, failureCallback);
-

✓ Use of catch() in Promises

- Handles rejected promises (errors)
- Cleaner than second parameter in then()

```
promise.catch(error => console.log("Error:", error));
```

✔ Use of finally() in Promises

- Runs **no matter what**, after either then or catch
- Useful for cleanup (like hiding a loader)

promise

```
.then(() => {})  
.catch(() => {})  
.finally(() => console.log("Always runs"));
```

🧠 Real-Life Analogy for Promises:

“Imagine ordering food. While it’s cooking (pending), you can do other things. When it’s done (fulfilled), you eat. If the order fails (rejected), you get a refund. Regardless, you clean the table (finally).”

◆ async / await

✔ Use of async and await

- async makes a function return a Promise
- await pauses code execution inside an async function until the Promise resolves

```
async function getData() {  
  let res = await fetch("data.json");  
  let data = await res.json();  
  console.log(data);  
}
```

Real-life analogy:

You "await" the waiter to bring food before you start eating. The rest of the meal (code) waits too.

✗ Can I use await in a function that is not async?

No!

You can only use await **inside an async function**. Otherwise, it throws a syntax error.

✔ Return type of fetch

→ Promise that resolves to a **Response object**

```
let res = await fetch("url"); // res is a Response object
```

✔ How many parameters does fetch accept?

fetch(url, options)

- url (string) – required
- options (object) – optional

```
fetch("https://api.example.com", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ name: "John" })  
});
```

✓ **How to serialize a JS object (convert to string)**

```
let jsonStr = JSON.stringify({ name: "John" });
```

✓ **Convert JSON string to JS object**

```
let obj = JSON.parse('{"name":"John"}');
```

✓ **How many stages to receive final data from fetch?**

1. `await fetch(...)` – gets **Response** object
 2. `await response.json()` – gets actual **data**
-

✓ **When fetch resolves, what do you get?**

→ A **Response object**, even if it's an error like 404!

✓ **When response resolves, what do you get?**

→ Actual **data** or a **parsing error**

```
let res = await fetch("...");  
let data = await res.json(); // or .text(), etc.
```

✓ **What is response.ok?**

- Returns true if status is in **200–299** range
- Helps check if fetch succeeded

```
if (response.ok) {  
  // success  
}
```

✓ **Read data from response**

```
await response.json(); // parse JSON  
await response.text(); // get as plain text  
await response.blob(); // for files  
await response.formData(); // for form submissions  
await response.arrayBuffer(); // for binary data
```

✓ **Default method for fetch**

```
fetch("url") // method = GET by default
```

✓ **Can we replace .then() with await?**

Yes!

- `await` is a cleaner, more readable way to work with Promises

// Using then

```
fetch("url").then(res => res.json()).then(data => console.log(data));
```

// Using await

```
let res = await fetch("url");  
let data = await res.json();  
console.log(data);
```

✅ **FormData().entries()**

Returns an **iterator** of all key-value pairs (like array)

```
let fd = new FormData(formElement);
for (let [key, value] of fd.entries()) {
  console.log(key, value);
}
```

✅ **Use of FormData**

- Helps to **read or send** form data as key-value pairs
- Used for file uploads, form submission via JS

```
let formData = new FormData(document.getElementById("myForm"));
```

✅ **Property name of form fields should match?**

→ Yes, it should match your **server-side model (POCO)** or field name

```
<input name="username" />
```

This will be sent as:

```
FormData = { username: "value" }
```

🔙 **Summary Cheat Tips for Viva**

Concept	Key Info
async/await	Cleaner Promise handling
fetch()	Promise → Response object
JSON.stringify	JS to JSON string
JSON.parse	JSON string to JS
FormData	Reads form as key-value
response.ok	HTTP status check (true/false)
await outside async	❌ Not allowed

◆ URLSearchParams(new FormData(form))

✓ What does it do?

Converts form data into a **query string** format like key=value&key2=value2.

```
let params = new URLSearchParams(new FormData(form));
```

```
console.log(params.toString());
```

```
// Example: age=25&name=Vita+Nidhi
```

✓ When should you use it?

- When sending form data via GET or POST (with content type: application/x-www-form-urlencoded)

◆ Collection Classes in JS

✓ Map

- Stores **key-value pairs**
- Keys can be any type (even objects)
- Keys are **unique** — new value **overwrites** the existing one

```
let map = new Map();
```

```
map.set("name", "Vita");
```

```
map.get("name"); // Vita
```

✓ Set

- Stores **unique values only**
- No key-value structure, just values

```
let set = new Set();
```

```
set.add("JS");
```

```
set.add("JS"); // Won't add again
```

```
set.has("JS"); // true
```

◆ Regular Expressions (Regex)

✓ Flags

- i – case-insensitive
- g – global (match all, not just first)

```
let regex = /a+/ig;
```

✓ Regex Patterns

Symbol	Meaning
--------	---------

*	0 or more times
---	-----------------

+	1 or more times
---	-----------------

?	0 or 1 time (optional)
---	------------------------

.	Any single character
---	----------------------

[0-9] or \d	Digit
-------------	-------

\D	Non-digit
----	-----------

[a-zA-Z]	Alphabet only
----------	---------------

{10}	Exactly 10 times
------	------------------

Symbol	Meaning
{2,5}	Between 2 to 5 times
^	Start of string
\$	End of string
\w	Word character [a-zA-Z0-9_]
\W	Non-word character

✅ Regex Methods

- `match()` → returns array of matches
- `search()` → returns index of first match
- `replace()` → replaces match with string

`"abc123".match(/\d+/); // ["123"]`

`"abc123".search(/\d+/); // 3`

`"abc123".replace(/\d+/, "****"); // abc***`

💡 Why React?

✅ Key Benefits:

- **Reusable Components** – Code modularly using components
- **Virtual DOM** – Fast updates and rendering
- **Declarative UI** – Focus on what to show, not how
- **Ecosystem** – Strong developer tools and community

✅ Is React part of MVC?

Yes, **React is the "View"** layer in the **MVC (Model-View-Controller)** architecture.

🧠 Quick Revision Summary

Topic	Concept
URLSearchParams	Form to query string
Map	Key-value store, unique keys
Set	Unique values only
Regex	Pattern matching
React	Fast, component-based UI library
React & MVC	React is the View

◆ React & JavaScript Viva Explained

✓ What is Virtual DOM?

- A **lightweight copy** of the real DOM.
- React uses it to track changes and **update only what's needed**, improving performance.

🔴 **Real-world analogy:** Like editing a draft (Virtual DOM) before finalizing a document (Real DOM).

✓ Project Setup

```
npm create vite@latest  
cd project-name  
npm install  
npm run dev
```

✓ What is package.json?

- Configuration file with **metadata**, dependencies, scripts, etc.

```
{  
  "name": "my-app",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build"  
  },  
  "dependencies": {  
    "react": "^18.0.0"  
  }  
}
```

✓ What is a Component?

- A **reusable UI unit** in React.
- Functional or class-based.

```
function Hello() {  
  return <h1>Hello!</h1>;  
}
```

✓ What is JSX?

- **JavaScript + XML**
- Lets you write HTML **inside JavaScript**.

```
const element = <h1>Hello, World</h1>;
```

✓ export vs export default

```
export const A = () => {}  
export default B;
```

- export → Named (need exact import name)
- export default → Single default export per file

✓ Passing Data Between Components

Parent → Child via props:

```
<Child name="Vita" />
```

Child:

```
function Child({ name }) {  
  return <p>{name}</p>;  
}
```

✓ Object Destructuring with Default

```
const { name = "Guest" } = props;
```

✓ What is SWC?

- **Speedy Web Compiler**
- Replaces Babel in modern tools like Vite and Next.js
- Written in Rust → Very fast

✓ Arrow Function

```
const add = (a, b) => a + b;
```

- No own this
- Shorter syntax

✓ Shortcut: Create Function Component

- rfce → React Functional Component Export
- rafce → Arrow Function version

(requires React snippets extension in VS Code)

✓ What is the Diff Algorithm?

- Compares **old vs new virtual DOM**
- Only updates what **actually changed**

✓ First File Loaded

- **Browser:** index.html
- **React App:** main.jsx or main.tsx

✓ Data Flow Pattern (Flux)

- **Unidirectional data flow:** Parent → Child via props
- State lifts up

✓ JSX Comment Syntax

```
{/* This is a JSX comment */}
```

✓ Conditional Rendering

```
condition && <h1>Hello</h1>      // AND
```

```
condition ? <A /> : <B />        // Ternary
```

```
if (x) {...} else {...}          // Standard
```

◆ React Hooks

✓ useState()

```
const [count, setCount] = useState(0);
```

- Returns an **array with 2 values**: state & setter
 - Keeps track of state in functional components
-

✓ Why use useState?

- To manage **local state** inside components
-

✓ useEffect()

```
useEffect(() => {  
  // side effect here  
}, [dependency]);
```

- Runs after **render**
 - **Dependency array** controls when it re-runs
-

✓ useNavigate() (React Router)

- Redirect programmatically

```
let navigate = useNavigate();  
navigate("/home");
```

✓ useParams() (React Router)

- Read URL parameters

```
let { id } = useParams();
```

✓ Handle Form Submission & Update Object

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  setEmployee(prev => ({  
    ...prev,  
    [e.target.name]: e.target.value  
  }));  
};
```

✓ HTTP Request with fetch()

```
fetch(url, {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify(employee)  
});
```

✓ event.preventDefault()

- Prevents form from refreshing the page

✔ Clean Code Reminder

- Use consistent naming
- Break into small components
- Keep logic outside JSX
- Comment where necessary

📘 Final Cheat Summary Table

Concept	Description
Virtual DOM	React's efficient diffing UI layer
JSX	HTML inside JS
useState	Local state + setter
useEffect	Side effects (e.g. fetch)
useNavigate	Redirect via code
useParams	Read URL params
SWC	Super-fast compiler
export vs default	Named vs single default export
Destructuring	{ name = 'guest' }
FormData	Serialize form for network send
Comments in JSX	{/* comment */}
